



Nombre:

Grupo:

Pregunta 1:

- [] Una estructura de datos implementada sobre un array que soporte inserciones en cualquier posición es considerada al mismo tiempo como $O(n)$ y $\Omega(1)$
- [] Un dato almacenado en un vector dinámico que permanece en dicha estructura de datos a lo largo de todo su ciclo de vida nunca cambia su posición de memoria.
- [] La clase A tiene como atributo un mapa de punteros a objetos tipo B, entonces entre ellos puede existir una relación de composición.
- [] Una matriz definida como `(int a[3][5])` se almacena en una zona contigua de memoria.
- [] Si se utilizara un vector como contenedor para un árbol AVL almacenando los hijos del dato (i) en las posiciones $(2i)$ y $(2i+1)$, las inserciones y borrados se realizarían necesariamente en tiempo $O(n)$ aún utilizando

- vectores de apoyo.
- [] No es más eficiente implementar una cola dinámica usando una lista de STL (list) que utilizando un vector (vector).
- [] Si se introducen datos ordenados de forma ascendente en un AVL, el tipo de rotación que se realizaría siempre sería el caso 4.
- [] Si la tabla hash T1 tiene un $\lambda=0.8$ significa que tiene más casillas vacías que T2 con un $\lambda=0.7$
- [] Las operaciones siguientes: `it=miMap.find(7); (*it).first = 8;` son válidas, siendo `map<int, int> miMap;`
- [] Es posible que un árbol B de orden 20 tenga menos altura que otro árbol B con orden 22 albergando exactamente los mismos datos.

Pregunta 2:

Mostrar el proceso de inserción de los siguientes datos: 7, 2, 9, 14, 5 en dos tablas de dispersión A y B de tamaño 5 y dispersión por división, donde A implementa dispersión abierta y B dispersión cerrada con exploración cuadrática. Agrupar varios pasos cuando sea posible siempre que no perjudique la legibilidad del ejercicio.

Pregunta 3:

- Implementar la funcion `vector<T> ABB<T>::InordenNR( )`; capaz de devolver los datos de un árbol ABB en inorden de forma no recursiva en un vector de salida.
- Implementar la función `void ListaEnlazada<T>::inserta(Iterador<T> &it1, ListaEnlazada<T> &l2)`; que inserte en la lista simplemente enlazada `*this` en la posición siguiente a la apuntada por el iterador `it1`, todos los elementos de la lista `l2`. Supongamos las listas: `*this={10, 20, 30, 40, 50}` y `l2={31, 32, 33, 34}`, apuntando `it1` a 30. El resultado tras realizar la operación sería: `*this={10, 20, 30, 31, 32, 33, 34, 40, 50}`. No podrán usarse otras operaciones de la clase `ListaEnlazada<T>`.

Pregunta 4:

Implementar un sencillo gestor de memoria, similar a los gestores de memoria dinámica de los sistemas operativos. La idea básica de este gestor es preasignar un búffer de objetos de tamaño grande para posteriormente ir concediendo bloques dentro de este buffer según las necesidades. La ventaja de este esquema es la velocidad: se hace una única petición de memoria al principio que es gestionada por nuestro gestor. Los constructores/operaciones son las siguientes:

- *GestorMemoria()*: Crea en memoria dinámica un buffer de objetos del tipo T, de tamaño igual al indicado como argumento. El tamaño del búffer permanece inalterable hasta que el gestor de memoria es destruido. El destructor correspondiente eliminará el espacio ocupado por el búffer.
- *asignar()*: Devuelve un puntero a un nuevo bloque del búffer con el tamaño indicado. El gestor de memoria deberá mantener internamente una estructura de datos con todos los bloques de memoria asignados previamente dentro del búffer (posición de comienzo y tamaño). La operación usará esta estructura de datos para

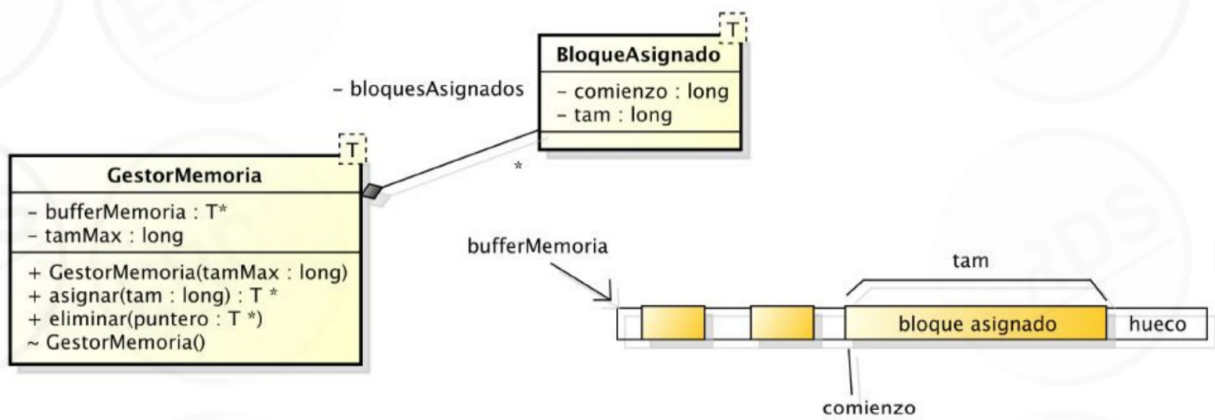


Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

localizar un hueco libre dentro del búffer con el tamaño igual o mayor al solicitado, creará una nueva entrada en la estructura de datos (*BloqueAsignado*) y devolverá un puntero al comienzo del bloque en el búffer. La estrategia a usar será usar el hueco disponible más pequeño que tenga el tamaño suficiente. Si no es posible hacer la asignación de memoria, se lanzará una excepción estándar C++ *bad_alloc*.

- *eliminar()*: Localiza el bloque asignado cuyo comienzo es apuntado por el puntero y lo elimina de la estructura de datos.

Se pide la implementación del gestor de memoria descrito, eligiendo la estructura de datos y los algoritmos adecuados. Puede utilizarse STL si se considera oportuno.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional